

MyDelivery End-to-End Functional Flow & Java 17 Modernization Plan

1. High-Level Architecture

Modules:

- Presentation (Spring WebFlow + Tiles + JSP/Freemarker + DWR): Interactive UI wizard for redelivery.
- WebService (Spring MVC REST): Programmatic endpoints for mobile/integration clients.
- ServiceImpl: Business logic, option calculation, validation orchestration, report building, external system ACLs.
- ServiceApi: Interfaces, enums, models, validation & email data contracts.
- Properties / Config / EAR: Packaging, environment configuration.

Cross-Cutting:

- Validation (OVal + custom annotations + helper classes).
- Reporting (JasperReports via StreamServe for form/confirmation PDFs).
- External Systems (APC, OSC, CommonCodes, StreamServe).
- Caching Aspect (AspectJ around read-only service calls).

2. Primary User Journey (WebFlow)

Entry: `/index.jsp` -> forward `/flow/redelivery.html` -> Spring WebFlow loads `redelivery-flow.xml` start-state `init`.

Flow States (simplified):

1. `init`: Decorator evaluates country display, consignment id display, determines applicability flags (postcoded country, RTL, double-byte). Decision to skip `enterDetails` if launched from email.
2. `enterDetails`: User supplies core consignment reference & country. Validation fires (bean validator). On continue: if errors -> same state else -> fetch available options.
3. `getAvailableOptions` (action-state): Calls `availableOptionsDecorator.getAvailableOptions()` -> underlying `DefaultMyDeliveryService` obtains consignment, runs validation rules, generates `DefaultAvailableOptionsModel`.
4. `redeliveryOptions`: User selects one of enumerated `RedeliveryOption` values.
5. Option-specific branch (`OriginalAddress` / `SelfCollection` / `AlternativeAddress` / `LeaveInAbsence` / `LeaveWithNeighbour`):
 - Decorator sets last successful step.
 - Fetch available dates (different strategy per option) using service methods / helper (`DefaultRedeliveryConsignmentHelper`, country metadata, depot availability etc.).

- Validate request context prior to rendering (ensures required core fields available).
- On continue: bind & validate option-specific form additions.
- 6. enterContactDetails (option-specific): Gathers contact (email, phone numbers—normalized & checked) + address/completion instructions.
- 7. confirm (option-specific confirm state): Decorator builds `RedeliveryRequestBean`, invokes `confirm()` which triggers:
 - Address/phone/email validation.
 - Redelivery request factory (`DefaultRedeliveryRequestFactory`) building immutable `DefaultRedeliveryRequest`.
 - Business rule checks: availability of selected date, allowed option presence, consignment state, country constraints, field length, forbidden characters.
 - Generation of `ConfirmationModel` and optionally `PrintPTLModel`.
- 8. confirmation page (confirmDetails): Displays success message, provides print actions for PTL or confirmation details.
- 9. Print actions fire externalRedirect to legacy servlets (`PrintMyDeliveryPTLFormServlet` / `PrintMyDeliveryConfirmationDetailsServlet`) invoking `MyDeliveryReportPrinter` -> JasperReports -> PDF stream.
- 10. end state: external redirect to tnt.com homepage.

3. WebService Flow (REST)

Endpoint invocation -> `MyDeliveryWebService` methods:

- Resolve & decrypt consignment id (`ConsignmentIdCryptor`).
- Parse/convert dates (`SimpleDateFormat`), locale determination.
- Build `ParamVO` -> validate (string emptiness, required IDs, option enumeration).
- Call `MyDeliveryService` methods to retrieve models (`AvailableOptionsModel`, `DatesModel`, `ConfirmationModel`).
- Wrap into `ResultVO` adding violations from underlying model.
- Serializes response (likely JSON though original stack may default to XML).

4. Core Business Logic Steps (`DefaultMyDeliveryService`)

1. Input sanitation: Trim, uppercase (country codes), null-safe conversions.
2. Consignment retrieval (APC ACL) & status evaluation (`DeliveryStatusCode` mapping).
3. Country metadata fetch (OSC Acl / `CommonCodes Acl`) -> rules (postcode supply, phone prefix sets, double-byte char allowance, option enablement flags).
4. Build `DefaultRedeliveryConsignment` via helper to enumerate permissible `RedeliveryOption` s:
 - Excludes options if blocked by country, consignment status (delivered / in transit), or missing required data (e.g. depot availability for `SelfCollection`).
5. Date availability generation per option (time windows, cut-off logic, earliest permitted date). Sorting & filtering in `DefaultDatesModel` / helper.
6. Validation layer: Field length checks (`FieldLengthConstants`), mandatory presence, format (email regex, phone numeric/prefix validation), neighbor/alternative address

completeness.

7. Factory creates `RedeliveryRequest` verifying:
 - Selected option in available set.
 - No outstanding validation errors.
 - Date still valid / not expired / within allowed window.
8. Confirmation: populates `DefaultConfirmationModel` (tracking code, sanitized contact, selected option label) & optionally triggers email sending (`EmailSender` interface) or prepares data for printing.
9. Printing: `PtlFormDataCreator` assembles `DefaultPtlFormData` -> Report printer or `StreamServe` ACL -> PDF.
10. Caching aspect: Wraps certain retrievals (country metadata, code lists) reducing external round trips.

5. Validation Checks Overview

Categories:

- Structural: Non-null, non-empty, length boundaries, allowed characters.
- Contact: Email format, phone prefix valid for country, mobile vs landline type presence.
- Address: Postcode rule (`SupplyPostcodeRule`) per country, required lines, double-byte character constraints.
- Option Eligibility: Option present in `AvailableOptionsModel`, date chosen in list, neighbor details if `LeaveWithNeighbour` selected.
- Temporal: Date not in past, within permitted range (e.g. next N days), depot date matches depot availability schedule.
- Consignment State: Not delivered already, not cancelled, not blocked by customs holds.
- Security: (Implied) encrypted consignment id decoding + potential tamper detection.

6. External Integrations

- APC: Consignment & address data.
- OSC: Operational service capabilities (country rules, service type metadata).
- CommonCodes: Reference data (country lists, option flags, localized labels maybe).
- StreamServe: Document generation (PTL form, confirmation labels) via XML payload marshalled (`RequestHandler` / `PayloadCreator`) and response parsing (`ResponseHandler`).

7. Reporting Flow (Print Actions)

1. User triggers print -> servlet invokes `ReportPrinter` or `StreamServeAcl`.
2. `MyDeliveryConfirmationDetailBuilder` builds value object; `MyDeliveryConfirmationDetailLabels` fetches localized keys using `ContentManagedMessageSource`.
3. `JasperReports` fill operation with assembled parameters & details -> bytes streamed as PDF.

8. Current Technical Debt / Legacy Indicators

- Java EE 2.4 web.xml + manual servlet declarations.
- Spring WebFlow XML definitions; heavy XML configuration (webflow-servlet.xml).
- AspectJ runtime weaving for caching (could migrate to Spring Cache abstraction).
- Use of SimpleDateFormat (thread-safety risk).
- Oval + custom validators rather than Bean Validation (Jakarta Validation) standard.
- Freemarker + Tiles mixture; JSP index forward.
- Manual encryption (ConsignmentIdCryptor) maybe replaceable by stronger libs.
- Lack of modularization for external system clients (could isolate clients in separate modules or use Feign / WebClient).

9. Java 17 Migration & Modernization Plan

Objectives

- Upgrade language level to Java 17 (LTS): leverage records, sealed classes, switch expressions, text blocks, improved concurrency.
- Replace legacy XML heavy configuration with annotation / Java config, ideally Spring Boot.
- Standardize validation & error handling.
- Introduce modern build & deployment pipeline (containerization).
- Improve test coverage and adopt modern testing frameworks (JUnit 5, Mockito, Testcontainers).

Strategy Phases

1. Assessment & Branching
 - Create migration branch; introduce Maven `maven.compiler.source/target = 17`.
 - Run baseline tests; fix compilation issues (e.g. illegal reflective access).
2. Dependency & Framework Upgrade
 - Migrate parent POM: update Spring to latest Spring Boot 3.x (Jakarta namespace). Decide whether to keep WebFlow (supported separately) or rewrite wizard logic using Spring MVC + state in session or React front-end.
 - Replace AspectJ caching with Spring Cache (Caffeine / Redis).
 - Move validation to Jakarta Validation (Hibernate Validator). Replace Oval annotations.
 - Replace Apache HttpClient usage with Spring WebClient (reactive) or OkHttp for simpler calls.
3. Module Consolidation
 - Create Boot application module combining Presentation & WebService (two endpoints layers) unless strict separation needed.
 - Retain ServiceApi as separate API (or convert to internal module + published Javadoc if consumed externally).
4. Configuration & Properties
 - Convert property files to Spring Boot `application.yml` profiles.
 - Externalize sensitive configs to environment variables / Vault.

5. Code Modernization

- Convert data carrier POJOs (Models) into Java 17 records where immutability suitable (`ConfirmationModel` , `DepotAddressModel` , etc.). Keep mutable builder-style objects as classes.
- Use sealed interfaces for restricted hierarchies (e.g., navigation steps).
- Replace builder patterns returning simple value objects with record factories.
- Introduce `java.time` API (`LocalDate` , `LocalDateTime` , `ZoneId`) eliminating `Date` & `SimpleDateFormat` .
- Implement switch expressions for `RedeliveryOption` handling.
- Utilize text blocks for XML payload templates (`StreamServe`) or consider JAXB removal replacing with Jackson XML.

6. Redelivery Flow Redesign (If Dropping WebFlow)

- Represent wizard state in a `RedeliverySession` record stored in HTTP session or persisted temporarily.
- Expose REST endpoints: `/redelivery/init` , `/redelivery/options` , `/redelivery/option/{type}/dates` , `/redelivery/confirm` .
- Client-side (SPA) controls navigation; server purely validates & returns state transitions.

7. Validation Layer Refactor

- Create annotation set (`@ValidPhone` , `@ValidPostcode(countryField=...)`) using Hibernate Validator custom constraints.
- Replace manual `ValidationHelper` loops with automatic Bean Validation; map violations to current `ViolationsModel` (or remove model if redundant).

8. External Client Modularization

- Wrap APC, OSC, CommonCodes, `StreamServe` in dedicated `client-*` packages using interfaces + implementation with resilience (retry, circuit breaker via `Resilience4j`).
- Provide test fakes & integration tests with `WireMock` / `Testcontainers`.

9. Observability

- Introduce structured logging (JSON) with `Logback`.
- Add metrics (`Micrometer`) & tracing (`OpenTelemetry`) for external calls.

10. Reporting Modernization

- Evaluate need for `StreamServe`. If retained, refactor to service with REST + JSON. If replaced, move `JasperReports` templates into classpath; use modern exporter APIs.

11. Security Enhancements

- Review `ConsignmentIdCryptor`; replace with AES/GCM or signed tokens (JWT) carrying consignment id & option list.
- Add input sanitization with Spring Boot's built-in features; enforce Content Security Policy.

12. Testing & Quality

- Migrate tests to JUnit 5 (update annotations).
- Add parameterized tests for option eligibility logic.
- Implement mutation testing (PIT) for core business logic.
- Add CI pipeline (GitHub Actions / Jenkins) with static analysis (`SpotBugs`, `Checkstyle`) & dependency vulnerability scanning.

13. Deployment

- Build container image (Docker) – multi-stage (mvn build then jre17 distroless).
- K8s or cloud runtime configuration (Helm charts). Externalize caches (Redis) & config.

14. Cutover & Data Integrity

- Parallel run old & new endpoints; compare responses using automated contract tests.
- Canary release for subset of traffic.
- Decommission legacy WebFlow/Tiles once parity achieved.

Work Breakdown (Epic -> Stories)

Epics:

- E1: Build & Config Migration.
- E2: Validation Refactor.
- E3: Flow Redesign (UI/API).
- E4: External Client Modernization.
- E5: Model & Domain Modernization (records/time API).
- E6: Reporting & Printing Upgrade.
- E7: Security & Observability.
- E8: Testing & Quality Gates.
- E9: Deployment & Cutover.

Each epic decomposes into small, verifiable stories (list not exhaustive):

- Replace Date with LocalDate (OriginalAddress path).
- Implement RedeliveryOption handling via switch expression.
- Introduce `@ValidAlternativeAddress` constraint.
- Migrate printing servlet to REST controller returning application/pdf.
- Introduce caching via Spring Cache (Caffeine) with metrics.

Risk Mitigation

- Maintain backward-compatible REST endpoints (versioned /v1 vs /v2).
- Feature flags for new flow activation.
- Shadow validation: run new validation layer alongside old until stable.

Success Metrics

- 0 critical CVEs in dependency list.
- 25% reduction in response time for available options call.
- 100% business rule parity (automated diff tests).
- Improved test coverage >85% lines, >70% branches.

10. Suggested Target Stack (Post-Migration)

- Java 17 + Spring Boot 3.x.
- Jakarta Validation + Hibernate Validator.
- Spring Cache + Caffeine/Redis.
- Web layer: REST + (optional) GraphQL for options/dates queries.
- Front-end: Modern SPA (React/Vue) or continue server-side if required.

- Observability: Micrometer + OpenTelemetry.
- Build: Maven + Jib or Dockerfile.

11. Immediate Quick Wins

- Replace SimpleDateFormat with DateTimeFormatter (thread-safe) even before full migration.
- Introduce unit tests for `DefaultRedeliveryConsignmentHelper` edge cases (country-specific option suppression).
- Centralize error code mapping to an enum implementing `IEnumErrorCode` + translator to HTTP statuses.

12. Decommissioning Steps

- Mark legacy servlets `@Deprecated`; route print endpoints to new REST controllers.
 - Remove Tiles definitions after SPA adoption; keep Freemarker only if email template rendering required.
 - Drop AspectJ weaving after cache refactor; remove aspect config from build.
-

Use this document as a living artifact; adjust stories & priorities after initial spike (running service under Java 17 without modernization) to surface incompatibilities early.